

Scientific Literature Intelligence Platform

Project Proposal for UCS503P

Submitted to: Ms. Paramveer Kaur

Thapar Institute of Engineering and Technology

Paarth Ganesh, CSED*

Diya Jain, CSED†

August 13, 2026

1 Higher-order goal

This project builds a retrieval-augmented research assistant that lets a user ask natural-language questions over a corpus of scientific papers and receive answers that are grounded in, and cited back to, specific source passages. The corpus is populated both automatically—by fetching papers from the arXiv API—and manually, by letting users upload their own PDFs. The engineering objective is a deployed web service that, given a question, returns an answer where every factual claim is traceable to a retrieved chunk, plus per-paper and cross-paper summaries. We treat retrieval quality and answer faithfulness as first-class, measurable properties rather than afterthoughts.

2 Time-to-value

- **Iteration 1 (end of week 2):** a working vertical slice—ingest a fixed set of arXiv PDFs, embed them, and serve semantic search over a REST endpoint. This alone is demonstrable and measurable.
- **Iteration 2 (week 4):** grounded question answering with inline citations, backed by a labeled evaluation set so every later change is scored, not guessed.
- **Iteration 3+ (week 5 onward):** summarization, structured knowledge extraction, hybrid retrieval, and reranking—each added only after it beats the previous version on the eval harness.
- CI runs the test suite and a retrieval-quality check on every commit, so regressions surface immediately rather than at demo time.

3 Problem Statement

Researchers, students, and analysts lose significant time reviewing literature across thousands of papers. The concrete failures we target are:

- **Information overload:** the volume of publications makes exhaustive manual review infeasible, so relevant prior work is missed.

*Roll No: 1024030171, Email: pganesh_be24@thapar.edu

†Roll No: 1024031154, Email: djain2_be24@thapar.edu

- **Keyword-search limitations:** lexical search fails when authors describe the same concept with different terminology, hiding semantically related work.
- **Ungrounded answers:** general-purpose chatbots fabricate citations and DOIs, so their output cannot be trusted for factual, source-backed claims.
- **Slow synthesis:** manually extracting and comparing methods, datasets, and reported metrics across many papers is repetitive and error-prone.

Why this matters now: scientific output is growing faster than any individual can absorb, while retrieval-augmented generation has made grounded, citation-backed answering practical on commodity hardware. The platform converts that capability into saved review time and verifiable answers.

4 Proposed Solution

4.1 Overview

A **web-based** platform composed of five subsystems:

- **Ingestion:** fetch papers from the arXiv API (or accept uploaded PDFs), extract text and metadata, chunk, embed, and store.
- **Semantic search:** embedding-based retrieval that surfaces conceptually relevant passages rather than exact keyword matches.
- **Grounded question answering:** answers generated strictly from retrieved passages, each claim carrying an inline citation to its source paper and page.
- **Summarization:** concise summaries of a single paper or across a set of retrieved results.
- **Knowledge extraction:** structured pull of task, method, dataset, and headline metric, enabling side-by-side comparison across papers.

4.2 Concrete technology stack

Chosen for a cost-aware, industry-realistic build—local models on the high-volume retrieval path, a hosted LLM only for final generation.

- **Backend:** Python + FastAPI (async, typed, auto-generated OpenAPI docs).
- **Frontend:** React with a lightweight component library; chat-style Q&A, search, and corpus views.
- **Embeddings (local):** `sentence-transformers all-MiniLM-L6-v2` (384-dim), run locally so retrieval has zero per-query API cost.
- **Vector store + database:** PostgreSQL with the `pgvector` extension—one system for both relational metadata and vector search, avoiding a second datastore.
- **Generation LLM (hosted):** Gemini 2.5 Flash, called only to compose the final grounded answer and summaries from retrieved context.
- **PDF parsing:** PyMuPDF for text and page-level positions (needed for page-accurate citations).
- **Ingestion source:** the official arXiv API for metadata and PDF links.
- **Deployment:** Docker Compose; app on Render/Railway, managed Postgres alongside.
- **CI/CD:** GitHub Actions—lint, tests, and a retrieval-quality gate on every push.

4.3 Core workflow

1. User fetches papers by arXiv query or uploads PDFs; the ingestion pipeline parses, chunks, embeds, and stores them.
2. User poses a natural-language question.
3. The retriever embeds the query and pulls the top- k chunks by vector similarity, optionally reranked.
4. The generator composes an answer strictly from those chunks, attaching a citation (paper + page) to each claim, and returns it with summaries and extracted fields.

5 Solution Approach

This is an engineering project: the novelty is in system design, retrieval quality, and grounding discipline, not in inventing models.

5.1 Ingestion and chunking

- Parse PDFs with PyMuPDF, retaining page numbers so every chunk knows its origin.
- Chunk by a fixed token window (≈ 500 tokens) with $\approx 15\%$ overlap, so context is not severed mid-idea.
- Store each chunk with its paper id, page, character span, and 384-dimensional embedding.
- Ingestion is idempotent: re-fetching a known arXiv id updates rather than duplicates it.

5.2 Retrieval and grounding

- Baseline: dense retrieval via cosine similarity on `pgvector` using an HNSW index for sub-linear search.
- Upgrade: hybrid retrieval—combine dense scores with lexical BM25, then rerank the merged top results with a cross-encoder for precision.
- Grounding contract: the generator receives only retrieved chunks and is instructed to answer solely from them; if the context does not support an answer, it says so rather than inventing one.
- Every claim in the output is tied to a chunk id, which resolves to a paper-and-page citation shown to the user.

5.3 Data model

Four core tables:

- `papers(id, arxiv_id, title, authors, year, source, created_at)`
- `chunks(id, paper_id, page, char_start, char_end, text, embedding vector(384))`
- `queries(id, user_question, created_at, latency_ms)`
- `answers(id, query_id, answer_text, cited_chunk_ids, groundedness_flag)`

5.4 CI/CD

GitHub Actions on every push:

- Lint (ruff) and format check, then unit + integration tests (pytest).
- A retrieval-quality gate: run the labeled eval set and fail the build if Recall@5 drops below the current threshold.
- Build the Docker image and deploy to staging on a green main branch.

6 Evaluation Criterion

Every metric is computed automatically against a hand-labeled evaluation set of question–passage pairs, so scores are reproducible and comparable across iterations.

6.1 Primary evaluation metric

Retrieval relevance — Recall@k: the fraction of evaluation queries for which at least one truly relevant passage appears in the top- k retrieved chunks.

- Target: Recall@5 ≥ 0.80 on the labeled set during the pilot.
- Attribution: measured directly against the ground-truth labels; the same harness runs in CI, so any regression is caught on commit.

6.2 Secondary evaluation metrics

- **Answer groundedness / faithfulness:** percentage of answer claims that are supported by a cited chunk, checked by an LLM-as-judge pass plus a spot-checked human sample.
- **Mean Reciprocal Rank (MRR):** rewards ranking the correct passage higher, not just including it.
- **Time-to-insight reduction:** time to locate a relevant paper on the platform versus a manual keyword-search baseline.
- **End-to-end latency:** p95 query latency ≤ 3 seconds, logged per query.
- **Reliability:** $\geq 99\%$ uptime for search and Q&A during the pilot window.

6.3 Pilot validation plan

- Assemble a fixed corpus of a few hundred arXiv papers in one domain (e.g., NLP) and hand-label ~ 50 questions with their supporting passages.
- Recruit 10–20 pilot users; compare time-to-insight and answer quality against the manual baseline using the labeled set and short interviews.

7 Scalability

1. **Scalable (theoretically):** the system absorbs growth in corpus size and concurrent queries by
 - using an HNSW index in `pgvector` for sub-linear nearest-neighbor search,

- decoupling ingestion (batch) from serving (online) so heavy embedding jobs never block queries,
- keeping the API stateless behind a load balancer.

2. **Operationally deployable (practically):** the system runs on common hosting via

- a single managed Postgres instance for both metadata and vectors,
- a containerized app deployable to Render/Railway,
- a GitHub Actions pipeline for staging and production.

8 Engine availability heuristic

Every component maps to a mature, available tool, so effort goes into system design and retrieval quality rather than infrastructure:

- FastAPI for the API layer; React for the UI.
- `sentence-transformers` for local embeddings; Gemini 2.5 Flash for generation.
- PostgreSQL + `pgvector` for combined relational and vector storage.
- PyMuPDF for parsing; the arXiv API for ingestion.
- pytest for testing; GitHub Actions for CI/CD; Docker for packaging.

9 Project scope and deliverables

9.1 Initial deliverable

- Ingestion pipeline: arXiv fetch + PDF upload, parse, chunk, embed, store
- Semantic search endpoint returning ranked chunks with paper/page metadata
- Grounded Q&A returning answers with inline source citations
- Labeled evaluation set and an automated harness computing Recall@k and MRR
- CI: lint + tests + retrieval-quality gate; CD to staging

9.2 Subsequent deliverables

- Per-paper and cross-paper summarization
- Structured knowledge extraction (task, method, dataset, metric)
- Hybrid retrieval (dense + BM25) with cross-encoder reranking
- Corpus management, filtering, and query-history views

10 Risks and mitigations

- **Hallucination / ungrounded answers:** strict retrieval-only prompting, mandatory citations, and an automated faithfulness check on every response.
- **Poor retrieval quality:** guard with the eval harness; escalate from dense to hybrid + reranking only when the data shows a gain.
- **PDF parsing failures:** PyMuPDF with a fallback path and per-document logging so bad files are isolated, not fatal.
- **Copyright / licensing:** restrict fetching to arXiv (open access) plus user-uploaded content.
- **Cost overrun:** embeddings run locally; the hosted LLM is called only at generation time, on a small budget.

11 Summary

This project delivers a deployable retrieval-augmented platform for scientific literature: arXiv and user-PDF ingestion, local embeddings with `pgvector` search, and Gemini-generated answers that cite their sources. It meets the course criteria through early time-to-value via a working vertical slice, CI/CD with an automated retrieval-quality gate for safe frequent releases, and rigorous measurable evaluation—chiefly Recall@k and answer groundedness. The engineering focus is a well-designed, cost-aware, scalable system rather than research novelty.